

Simulationsergebnisse der Teststärke des multivariaten Kolmogorov-Smirnov Tests

von David Hamann

1 Das Theorem und der Algorithmus

Gegeben sei die Stichprobe $x_1, \dots, x_n$ von i.i.d. p -dimensionalen Zufallsvariablen mit Verteilung F . Für den Test $H_0 : F = F_0$ gegen $H_1 : F \neq F_0$ ist

$$D_n = \max_{j=1,2,\dots} d_n^j$$

die multivariate Version der Kolmogorov-Smirnov Teststatistik. Dabei ist $d_n^j = \sup_{y^j} |G_n(y^j) - y_1^j \cdots y_p^j|$ und

$$y_1^j = F(z_1^j)$$

$$y_i^j = F(z_i^j | z_{i-1}^j, \dots, z_1^j)$$

die Folge an Transformationen (Rosenblatt Transformation), die alle $(y_1^j, \dots, y_p^j)$ erzeugt. $(z_1^j, \dots, z_p^j)$ für $j = 1, \dots, p!$ ist die j -te Permutation von $(x_1, \dots, x_p)$.

Nach dem Artikel von Justel et al. kann diese Statistik im zweidimensionalen Fall durch folgende endliche Menge an Differenzen geschrieben werden:

$$D_n = \max_{u \in I, v \in P} \{G_n(u) - G(u), G(v) - G_n(v^-)\}.$$

Dabei sind $I = \{(x_j, y_i) | x_i \leq x_j, y_i \geq y_j; i, j = 0, 1, \dots, n\}$ und $P = \{(x_j, y_j) | x_j > x_i, y_j < y_i; i, j = 0, 1, \dots, n+1\}$, G ist die Verteilungsfunktion von zwei unabhängigen gleichverteilten Zufallsvariablen zwischen 0 und 1 und G_n ist die empirische Verteilungsfunktion. I ist die Menge des Punktes $(0, 0)$ und der intersection points (x_j, x_i) für $x_i < x_j$ und $y_i > y_j$. P ist die Menge des Punktes $(1, 1)$, der intersection points und der Projektionen der beobachteten Punkte auf den rechten und den oberen Rand des Einheitsquadrats.

In Figure 1 ist die Beschreibung der algorithmischen Implementation dieser finiten Differenzen aus dem Artikel.

As a consequence of Theorem 2, D_n may be obtained by evaluating the distance in a finite amount of points which ranks from $3n$ to $3n + \binom{n}{2}$ depending on the sample configuration. The theorem leads to the following procedure to compute the Kolmogorov-Smirnov statistic (2.4): (1) compute the maximum distance in the observed points, $D_n^1 = \max_{i=1,\dots,n} D_n^+(u_i)$; (2) compute the maximum and minimum distances in the intersection points, $D_n^2 = \max_{i,j=1,\dots,n} \{D_n^+(x_j, y_i) | x_j > x_i, y_j < y_i\}$ and $D_n^3 = 2/n - \min_{i,j=1,\dots,n} \{D_n^+(x_j, y_i) | x_j > x_i, y_j < y_i\}$; (3) compute the maximum distance among the projections of the observed points on the right unit-square border, $D_n^4 = 1/n - \min_{i=1,\dots,n} D_n^+(1, y_i)$; (4) compute the maximum distance among the projections of the observed points on the top unit-square border, $D_n^5 = 1/n - \min_{i=1,\dots,n} D_n^+(x_i, 1)$; and (5) compute the maximum $D_n = \max\{D_n^1, D_n^2, D_n^3, D_n^4, D_n^5\}$.

Figure 1: Beschreibung des Algorithmus im Artikel

Dabei ist $D_n^+(u) = (G_n(u) - G(u))$ und $D_n^-(u) = (G(u) - G_n(u))$. Figure 2 (siehe Anhang) ist der im Artikel beschriebene Algorithmus implementiert in Python. Dabei ist `empirical_cdf` die zweidimensional empirische Verteilungsfunktion, `G_uniform` die Verteilungsfunktion der zweidimensionalen Gleichverteilung auf 0 und 1 und `ks_2d_statistic` die zweidimensionale Kolmogorov-Smirnov Teststatistik, wie sie oben beschrieben ist. D1 bis D5 im Code korrespondieren direkt mit D_n^1 bis D_n^5 aus der Beschreibung des Algorithmus in Figure 1.

2 Monte-Carlo Simulation

Für eine gegebene Stichprobe lässt sich die Teststatistik D_n errechnen. Diese Statistik kann dann mit den Perzentilen der wahren Verteilung von D_n verglichen werden, um eine Entscheidung bezüglich der Hypothesen zu treffen. Die Perzentile der Verteilung von D_n lassen sich mithilfe von Monte-Carlo Simulation approximieren, da es keine geschlossene/analytische Form gibt. In Figure 3 (siehe Anhang) ist der Code zur Implementation der Monte-Carlo Simulation für $n = 15$ und $\alpha = 0.05$ zu sehen.

Eine Gegenüberstellung der Ergebnisse der Monte-Carlo Simulation für 2000 Durchläufe ist in Table 1 gegeben:

n	MC Perzentilwerte (Justel et al.)	eigene MC Perzentilwerte
15	0.4141	0.4118
25	0.3254	0.3277
50	0.2350	0.2350
100	0.1675	0.1663

Table 1: MC Simulationswerte im Vergleich

Die absolute Abweichung der Simulationen liegt in der Größenordnung 10^{-3} . Dies liegt im Rahmen der Abweichung, die erwartbar ist.

3 Simulation des Anpassungstests

Im Artikel werden unterschiedliche Simulationsergebnisse präsentiert. Unter anderem Ergebnisse in der zweidimensionalen Kolmogorov-Smirnov Teststatistik im Kontext eines Anpassungstests für Normalverteilung verwendet wird. Als H_0 wird die bivariate Normalverteilung mit $\mu = 0$ und Kovarianzmatrix

$$\Sigma = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$$

betrachtet. Die Alternativverteilung ist die Gaußsche Mischverteilung $(1 - \varepsilon)N_2(0, \Sigma) + \varepsilon N_2(\mu, \Sigma)$ für $\varepsilon \in \{0.1, 0.2, 0.4\}$ und $\mu = (3, 3)^T$. Da die Verteilung der H_0 bekannt ist, kann man Datenpunkte $x_1, \dots, x_n$ aus der Mischverteilung sampeln und diese mithilfe der Rosenblatt-Transformation wie folgt transformieren:

$$\begin{aligned} u_1 &= F_0(x_1) \\ u_i &= F_0(x_i | x_{i-1}, \dots, x_1) \end{aligned}$$

Mit den daraus erhaltenen Punkten $u_1, \dots, u_n$ kann die Kolmogorov-Smirnov Teststatistik $D_n = \sup_{u \in [0,1]^2} |G_n(u) - \prod u_i|$ mithilfe der `ks_2d_statistic` Funktion (siehe Anhang Figure 2) errechnet werden. Da unter der H_0

$$(X_1, X_2) \sim \mathcal{N}(0, \Sigma)$$

gilt, folgt, dass $X_1 \sim \mathcal{N}(0, 1)$ und $F_1(x_1) = \Phi(x_1)$. Da $X_1|X_2$ auch normalverteilt ist, folgt $X_2|X_1 = x_1 \sim \mathcal{N}(0.5x_1, 0.75)$ also $F_2(X_2|X_1) = \Phi(\frac{X_2 - 0.5X_1}{\sqrt{0.75}})$, da $\mathbb{E}[X_2|X_1 = x_1] = 0.5x_1$ und $\text{Var}(X_2|X_1) = 1 - 0.5^2 = 0.75$.

Die Implementierung des Anpassungstests mit 10.000 Durchläufen für $n = 15$ der oben genannten Gaußschen Mischung ist in Figure 4 (siehe Anhang) zu sehen.

Eine alternative Implementierung der Mischung ist in Figure 5 (siehe Anhang) gezeigt.

Eine Gegenüberstellung der Simulationsergebnisse des Artikels und meiner Simulationsergebnisse ist in Table 2 zu sehen. Die beiden Werte der Teststärke meiner Simulation sind das Ergebnis von zwei unterschiedlichen Durchläufen mit jeweils 10.000 Replikationen des Berechnens der Teststärke aus Stichproben der Gaußschen Mischung ($num_sim = 10.000$). Um die Ergebnisse des Ar-

tikels möglichst akkurat zu reproduzieren und die Monte-Carlo simulierten D_n Perzentile sich kaum unterscheiden, habe ich für die Simulation der Teststärke die Perzentilwerte des Artikels benutzt. Die Abweichung der Teststärke der Durchläufe meiner Simulation liegt im Bereich 10^{-3} bzw. hat eine maximale Abweichung von 0.01. Diese Abweichungen sind durch die Variation des Samplings zu erklären. Die Differenz zwischen den Werten der Teststärke im Artikel und meinen Werten beträgt allerdings bis zu 0.06 und deutet somit sehr eindeutig auf einen systematischen Fehler hin. Besonders, da die Replikation meiner eigenen Simulation nahe legt, in welchem Bereich eine nicht systematisch abweichende Teststärke liegen würde. Ein genereller Trend der Werte des Artikels ist allerdings auch in meinen Ergebnissen sichtbar: mit größerem ε und steigender Samplegröße erhöht sich die Teststärke. Auch sind die inkrementellen Unterschiede in der Teststärke bei Änderung von ε und n den Werten des Artikels sehr ähnlich, z.B. unterscheidet sich im Artikel die Power für $n = 15$ und $\varepsilon = 0.1$ zur Power für $n = 15$ und $\varepsilon = 0.2$ um 0.14. In meinen Simulationen ist der Unterschied bei 0.1315 bzw. 0.1217.

Trotz der systematischen Abweichung scheinen meine Teststärke Simulationen dennoch plausibel und auch die Veränderung bei Variation von n und ε bleibt erhalten.

Auf der Suche nach Fehlern habe ich folgende potenzielle Fehlerquellen überprüft. Ich habe die Rosenblatt Transformation, zusätzlich zur oben beschriebenen Variante, mit der Definition der bedingten Dichte nachgerechnet, den Algorithmus für die Teststatistik mehrmals mit der Beschreibung im Artikel abgeglichen und nach eigenen Fehlern im Code abgesucht. Des Weiteren habe ich eine zweite Variante einer Gaußschen Mischung implementiert (siehe Figure 4 und 5).

n	ε	Teststärke (Justel et al.)	Teststärke (eigene Werte)
15	0.1	0.13	0.0817/0.0918
	0.2	0.27	0.2132/0.2135
	0.4	0.73	0.6738/0.6706
25	0.1	0.16	0.1155/0.112
	0.2	0.40	0.335/0.335
	0.4	0.92	0.8899/0.8863
50	0.1	0.23	0.177/0.1712
	0.2	0.67	0.6027/0.5984
	0.4	1	0.996/0.9958
100	0.1	0.41	0.3198/0.3119
	0.2	0.94	0.9216/0.9176
	0.4	1	1/1

Table 2: Vergleich der simulierten Power Werte

A Anhang

```
def empirical_cdf(x, y, X, Y):
    return np.mean((X <= x) & (Y <= y))

def G_uniform(x, y):
    return x * y

def ks_2d_statistic(X, Y, G):

    n = len(X)
    X = np.clip(X, 0, 1)
    Y = np.clip(Y, 0, 1)

    # maximum of observed points
    D1 = max(
        empirical_cdf(X[i], Y[i], X, Y) - G(X[i], Y[i])
        for i in range(n)
    )

    # maximum distance over all intersection points
    D2 = max(
        empirical_cdf(X[j], Y[i], X, Y) - G(X[j], Y[i])
        for j in range(n) for i in range(n)
        if (X[j] > X[i] and Y[j] < Y[i])
    )

    # minimum distance over all intersection points (with 2/n correction)
    D3 = (2 / n) - min(
        empirical_cdf(X[j], Y[i], X, Y) - G(X[j], Y[i])
        for i in range(n) for j in range(n)
        if (X[j] > X[i] and Y[j] < Y[i])
    )

    # maximum distance among projections of observed points on the right boundary (x = 1)
    D4 = (1/n) - min(
        empirical_cdf(1, Y[i], X, Y) - G(1, Y[i])
        for i in range(n)
    )

    # maximum distance among projections of the observed points on top boundary (y = 1)
    D5 = (1/n) - min(
        empirical_cdf(X[i], 1, X, Y) - G(X[i], 1)
        for i in range(n)
    )

    # final statistic
    Dn = max(D1, D2, D3, D4, D5)

    return Dn
```

Figure 2: Zweidimensionale K-S Teststatistik implementiert in Python


```
# alpha quantiles
n = 15
np.random.seed(10)
Dn_null = []
for _ in tqdm(range(10000)):
    sample = np.random.uniform(0, 1, size=(n, 2)) #sample from two dim. uniform 0-1
    X0, Y0 = sample[:,0], sample[:,1] # split x and y in respective vector
    Dn_null.append(ks_2d_statistic(X0, Y0, G_uniform))
c_alpha = np.quantile(Dn_null, 0.95)
```

Figure 3: Implementierung der Monte-Carlo Perzentilsimulation

```
num_sim = 10000

mean0 = np.array([0,0])
mean1 = np.array([3,3])
cov = np.array([[1, 0.5], [0.5, 1]])

c_alpha = 0.4141
power_values = []
n = 15
epsilon_list = [0.1, 0.2, 0.4]
for epsilon in epsilon_list:
    Dn_list = []
    for k in tqdm(range(num_sim)):
        # for each observation, decide whether it comes from
        # component 1 or component 0
        components = np.random.rand(n) < epsilon
        samples = np.zeros((n, 2))

        # number of observations from each component
        n1 = np.sum(~components)
        n2 = np.sum(components)

        # generate observations from N(mean0, cov)
        samples[~components] = np.random.multivariate_normal(mean0, cov, n1)

        # generate observations from N(mean1, cov)
        samples[components] = np.random.multivariate_normal(mean1, cov, n2)

        X1 = samples[:,0]
        X2 = samples[:,1]

        # Rosenblatt transform
        U1 = norm(loc=0, scale=1).cdf(X1)
        U2 = norm(loc=0.5*X1, scale=np.sqrt(0.75)).cdf(X2)

        Dn_list.append(ks_2d_statistic(U1, U2, G_uniform))

    # power estimation:
    # proportion of simulations where the KS statistic
    # exceeds the critical value
    power = np.mean(np.array(Dn_list)>c_alpha)
    power_values.append(power)
```

Figure 4: Anpassungstest-Loop mit 10.000 Durchläufen

```

num_sim = 10000

mean0 = np.array([0,0])
mean1 = np.array([3,3])
cov = np.array([[1, 0.5], [0.5, 1]])

c_alpha = 0.4141
power_values = []
n = 15
epsilon_list = [0.1, 0.2, 0.4]
for epsilon in epsilon_list:
    Dn_list = []
    for k in tqdm(range(num_sim)):
        # alternative distribution
        gmm = GaussianMixture(n_components=2)
        gmm.weights_ = np.array([1-epsilon, epsilon])
        gmm.means_ = np.array([mean0, mean1])
        gmm.covariances_ = np.array([cov, cov])
        gmm.precisions_cholesky_ = np.linalg.cholesky(np.linalg.inv(cov))\
            [None, :, :].repeat(2, axis=0)
        samples, _ = gmm.sample(n)

        X1 = samples[:,0]
        X2 = samples[:,1]

        U1 = norm(loc=0, scale=1).cdf(X1)
        U2 = norm(loc=0.5*X1, scale=np.sqrt(0.75)).cdf(X2)

        Dn_list.append(ks_2d_statistic(U1, U2, G_uniform))

    power = np.mean(np.array(Dn_list)>c_alpha)
    power_values.append(power)

print("power values:", power_values)

```

Figure 5: Alternativer Anpassungstest-Loop mit 10.000 Durchläufen